

**Isha Training Solutions**CCAR-P • CLAUDE CERTIFIED ARCHITECT
PROFESSIONAL MOCK EXAMS

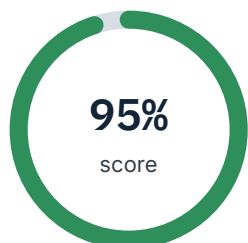
Overview

Mock
Exams

+91-9133190573

Start a mock exam

Independent, unofficial exam prep by Isha Training Solutions – not affiliated with,
endorsed by, or sponsored by Anthropic.

[Home](#) / [AI Courses](#) / [CCAR-P: Claude Certified Architect Professional Mock Exams](#)**952**/₁₀₀₀

20 of 21 correct on Mock Exam 1

Retake this exam

Back to mock exams

Review your answers

✔ **Correct** • Integration

A knowledge base is large, changes daily, and only a small relevant slice is needed for any given question. Which architecture fits better than stuffing the whole corpus into context?

- ☐ A Long-context stuffing of the entire corpus into every request.
- ☒ B Retrieval-augmented generation (RAG), pulling in only the relevant slice at query time. Correct answer
- ☐ C Ignoring the knowledge base entirely and relying on the model's general training knowledge.
- ☐ D Hard-coding a fixed subset of documents into the system prompt permanently.

WHY

RAG is architected for exactly this: a large, frequently changing corpus where only a relevant slice matters per query, rather than a small, static reference that could reasonably be stuffed into context

directly.

✔ Correct • Integration

A team is tuning chunk size for their retrieval pipeline. What's the core tradeoff they're balancing?

- ☐ A Chunk size only affects storage cost, never retrieval quality.
- ☒ B Chunks too large dilute relevance by mixing in unrelated content, while chunks too small lose surrounding context — the right size depends on the content's structure. Correct answer
- ☐ C There is no tradeoff; larger chunks are strictly better in every case.
- ☐ D Smaller chunks always retrieve faster with no downside.

WHY

Chunking is a real tradeoff: oversized chunks pull in irrelevant material alongside the useful part, while undersized chunks can strip away context needed to make sense of what was retrieved — tuning depends on the content itself.

✔ Correct • Integration

A single internal system needs to be consumed by several different Claude-powered clients across the organization. What consideration should drive the choice between exposing it via MCP versus building direct point-to-point integrations for each client?

- ☒ A Whether the integration needs to be reused across multiple independent clients — MCP's overhead pays off with reuse and independent maintainability, while a single simple one-off consumer may not need it. Correct answer
- ☐ B The choice should be made randomly since both are equivalent.
- ☐ C Point-to-point integrations are always architecturally superior.
- ☐ D MCP can only ever be used by one client at a time.

WHY

This is a classic service-boundary decision: a shared, well-scoped MCP server earns its overhead

when multiple independent clients need the same capability, whereas a single simple consumer may be better served by a direct integration.

✔ Correct • Integration

A production multi-agent pipeline is difficult to debug because failures could originate at any of several tool calls or model invocations. What architectural investment addresses this?

- ☐ A Adding more agents to the pipeline.
- ☐ B Removing logging entirely to reduce noise.
- ☒ C Instrumenting the pipeline with tracing/logging across every tool call and model invocation, not just the top-level request and response.
- ☐ D Relying solely on user complaints to detect failures.

Correct answer

WHY

Observability at the level of individual tool calls and model invocations — not just the overall request/response — is what actually lets a complex pipeline's failures be traced back to their real origin.

✔ Correct • Solution Design and Architecture

A team is about to design a complex multi-agent orchestration system, but a closer look suggests the real bottleneck is inconsistent, poor-quality source data. What should happen before committing to the multi-agent design?

- ☒ A Identify the dominant underlying problem first — here, data quality — since solving that may matter more than any orchestration pattern chosen on top of it.
- ☐ B Assume orchestration complexity always fixes data quality issues.
- ☐ C Skip architecture entirely and let engineers improvise.
- ☐ D Proceed with the multi-agent architecture regardless, since more agents generally help.

Correct answer

WHY

Architecture should follow from a clear-eyed diagnosis of the actual bottleneck. Adding orchestration complexity on top of a data-quality problem doesn't fix the real issue and can add cost and fragility for little benefit.

✔ **Correct** • **Solution Design and Architecture**

A solution needs to serve many concurrent users across multiple server instances. What design principle supports horizontal scalability here?

- ☐ **A** Limiting the system to a single server instance permanently.
- ☐ **B** Storing each user's session state only in the memory of whichever server instance first handled them.
- ☒ **C** **Designing request handling to be stateless, so any instance can serve any request without depending on state held elsewhere.** Correct answer
- ☐ **D** Statelessness has no relationship to horizontal scalability.

WHY

Stateless request handling is what allows multiple instances to interchangeably serve concurrent load — state pinned to one particular instance's memory undermines the ability to scale horizontally.

✔ **Correct** • **Solution Design and Architecture**

One component of a production system (a third-party enrichment service) starts failing intermittently. What design pattern keeps this from cascading into a full outage?

- ☐ **A** Ignoring the failing component and hoping it self-resolves.
- ☐ **B** Merging all components into a single tightly-coupled process to simplify the architecture.
- ☒ **C** **Isolating the failure domain with patterns like circuit breakers and timeouts, so one component's trouble doesn't bring down the rest of the system.** Correct answer
- ☐ **D** Removing all timeouts so requests wait indefinitely for the failing component.

WHY

Failure isolation techniques — timeouts, circuit breakers, and similar patterns — keep one component's problems from cascading through the whole system, which is exactly the risk of a tightly coupled design.

⊗ Incorrect · Solution Design and Architecture

Which of the following help future maintainers understand a non-obvious architectural tradeoff, such as choosing eventual consistency over strong consistency for a data path? (Select all that apply.)

☐ **A** Relying solely on the original architect's memory, with no written record.

☒ **B** Explaining the specific tradeoffs considered and why this option was chosen over the alternatives.

Correct answer

☒ **C** Recording the decision and its reasoning, such as in an architecture decision record.

Correct answer

☐ **D** Designing request handling to be stateless wherever that's a separate, unrelated concern.

Your answer

WHY

Writing down both the decision and the reasoning behind it (an ADR-style record) is what actually transfers understanding to future maintainers. Relying on memory alone leaves no record once the original architect moves on, and statelessness is a separate, unrelated design concern.

✔ Correct · Evaluation, Testing and Optimization

A team wants to validate a new model version against the current one before fully switching over. What approach lets them do this with real usage data while limiting risk?

☐ **A** Only test internally with team members and skip real user traffic entirely.

☒ **B** Run an A/B test against live traffic with a clear success metric, comparing the new version to the current one before a full rollout.

Correct answer

☐ **C** Switch every user to the new version immediately with no comparison.

- ☐ D Assume the newer version is automatically better with no testing.

WHY

A/B testing against real traffic, with a defined success metric, gives a grounded, risk-managed way to validate a change before committing to a full rollout, rather than assuming newer automatically means better.

✔ **Correct** • Evaluation, Testing and Optimization

A feature passed its evaluation before launch, but quality has never been re-checked since. What operational practice addresses the risk this creates?

- ☐ A Nothing further is needed once a feature has passed evaluation once.

- ☐ B Monitoring only matters for cost, not for quality.

- ☒ C **Continuous monitoring of quality, latency, and cost in production after launch, since real-world traffic and usage patterns can drift from what was originally evaluated.** Correct answer

- ☐ D Re-running the exact same pre-launch evaluation once a year is sufficient regardless of production signals.

WHY

Pre-launch evaluation is a snapshot; production traffic drifts over time. Ongoing monitoring of quality, latency, and cost is what catches regressions that a one-time evaluation, however thorough, can't anticipate.

✔ **Correct** • Evaluation, Testing and Optimization

A quality regression appears after a week where the prompt was tweaked, the model version was updated, and the retrieval index was also refreshed, all around the same time. What should happen before making further changes?

- ☐ A Immediately revert every change made that week without investigation.

- ☐ B Assume it must be the model version without checking anything else.

- ☐ C Ignore the regression since multiple changes happened simultaneously.

D

Isolate which specific change (prompt, model version, or retrieval data) is actually responsible before making further edits, rather than guessing.

Correct answer

WHY

With several concurrent changes, the sound approach is to isolate which one actually caused the regression (e.g., by testing each independently) before making further changes — guessing risks fixing the wrong thing.

✔ **Correct** • Governance, Safety and Risk Management

Which of the following reduce governance risk when many teams across an organization independently build their own Claude-powered integrations? (Select all that apply.)

A

Letting each team design its own access-control approach with no cross-team coordination.

B

Maintaining an audit trail for consequential automated decisions across the organization.

Correct answer

C

Establishing shared, organization-wide standards for security review and access control.

Correct answer

D

Agreeing on consistent tool-naming and scoping conventions across teams.

Correct answer

WHY

Shared review/access-control standards, consistent naming and scoping conventions, and organization-wide audit trails all reduce risk at scale. Leaving every team to design access control independently is exactly the uncoordinated pattern that creates inconsistent security gaps.

✔ **Correct** • Governance, Safety and Risk Management

A system automates both routine, low-stakes actions and rare, high-stakes actions (like large financial transfers). Should both categories have the same approval requirements?

A

No — higher-stakes actions should carry stricter human-in-the-loop approval requirements than low-stakes ones, with approval requirements tiered to risk.

Correct answer

B

No approval requirements are needed for any automated action.

- ☒ C Yes, identical approval requirements should apply to every action regardless of stakes.
- ☐ D Approval requirements should be based solely on how frequently an action occurs, not its stakes.

WHY

Risk-tiered approval — stricter checkpoints for higher-stakes actions and lighter-touch handling for low-stakes ones — matches the level of oversight to the actual consequences of getting something wrong.

✔ **Correct** • Governance, Safety and Risk Management

A newly deployed automated system starts behaving unexpectedly in production. What design element allows the team to respond quickly?

- ☐ A Waiting for the next scheduled release cycle to address the issue.
- ☐ B Relying entirely on the automated system to detect and fix its own misbehavior.
- ☒ C **A rollback or kill-switch mechanism that lets the team quickly disable or revert the system without a lengthy, high-risk emergency deployment.**
- ☐ D No such mechanism is needed if the system was well-tested before launch.

Correct answer

WHY

However well-tested a system is, production can surprise you. A fast, low-risk way to disable or roll back a misbehaving system is essential for limiting damage while the actual issue is investigated.

✔ **Correct** • Stakeholder Communication and Lifecycle Management

A business stakeholder needs to understand why a proposed architecture costs more but responds faster. How should this tradeoff be communicated?

- ☒ A **In terms of the business impact — such as the effect on cost, user experience, and outcomes that matter to the stakeholder — rather than purely technical terms.**
- ☐ B Using only technical jargon and implementation details with no framing toward business impact.

Correct answer

- ☐ C By asking the stakeholder to read the raw architecture diagram unassisted.
- ☐ D The tradeoff should not be communicated to business stakeholders at all.

WHY

Communicating a technical tradeoff to a non-technical stakeholder means translating it into what actually matters to them — business impact and outcomes — rather than assuming technical framing alone will land.

✔ **Correct** • Stakeholder Communication and Lifecycle Management

During a project kickoff, stakeholders assume the new Claude-powered feature will be perfectly accurate on every request. What should the architect do?

- ☐ A Promise perfect accuracy to keep stakeholders happy in the short term.
- ☐ B Let the assumption stand uncorrected to avoid a difficult conversation.
- ☐ C Avoid discussing system limitations at any point in the project.
- ☒ D Set realistic expectations upfront about what the system can and cannot reliably do, to avoid overpromising and a painful mismatch later.

Correct answer

WHY

Managing expectations honestly and early — being clear about real capabilities and limitations — prevents a much more damaging mismatch between promise and reality once the system is actually in production.

✔ **Correct** • Stakeholder Communication and Lifecycle Management

A new Claude-powered capability is ready for its first real users. What rollout approach best manages risk while still gathering real feedback?

- ☐ A Skipping any pilot or beta stage to move faster, regardless of risk.
- ☐ B Launching directly to the entire user base with no earlier stage.
- ☒ C A phased rollout — internal pilot, then limited beta, then full launch — to manage risk and incorporate feedback incrementally.

Correct answer

- ☐ D Rollout strategy has no bearing on risk management.

WHY

A phased rollout lets a team catch problems and incorporate feedback at a smaller scale before committing to a full launch, substantially reducing the risk compared to going straight to the entire user base.

✔ **Correct** • Claude Models, Prompting and Context Engineering

Multiple product surfaces share a lot of common system-prompt content but each also needs a bit of surface-specific customization. How should this be architected to avoid duplication?

- ☐ A Prompt architecture has no meaningful reuse strategy across product surfaces.
- ☐ B Maintain a fully separate, independently duplicated system prompt for each surface with no shared structure.
- ☐ C Combine every surface into one single undifferentiated system prompt with no per-surface distinction at all.

- ☒ D **Structure a shared base of common instructions that each surface extends or overrides with its own specific additions, rather than duplicating the whole prompt everywhere.** Correct answer

WHY

A shared base with surface-specific extensions avoids maintaining several near-duplicate prompts independently, so common updates propagate everywhere while each surface still gets its needed customization.

✔ **Correct** • Claude Models, Prompting and Context Engineering

A long-running session is expected to grow well beyond a comfortable context size over its lifetime. What context-engineering strategy should be designed in from the start?

- ☐ A Assuming the context window is unlimited and designing nothing for growth.
- ☐ B Waiting until a hard context-limit failure occurs, then reacting.

- ☒ **C Proactively designing for summarization/compaction of older turns as the session grows, rather than treating it as an edge case to handle only after something breaks.** Correct answer
- ☐ **D** Ending every session after a fixed, small number of turns regardless of the use case.

WHY

For sessions expected to run long, proactive context management (summarization, compaction) should be part of the initial design, rather than something bolted on reactively after a hard failure occurs.

✔ **Correct** • Claude Models, Prompting and Context Engineering

Which of the following help keep Claude's behavior consistent across a product built by multiple engineers? (Select all that apply.)

- ☒ **A Standardizing shared prompt templates and conventions across the team.** Correct answer
- ☐ **B** Using the most capable, most expensive model tier uniformly for every step regardless of its actual difficulty.
- ☒ **C Structuring a shared base of common system-prompt instructions that individual surfaces extend rather than duplicate.** Correct answer
- ☐ **D** Letting every engineer independently invent their own prompt structure with no shared convention.

WHY

Standardized templates and a shared base that surfaces extend (rather than duplicate) both promote consistency and maintainability. Letting every engineer improvise independently undermines consistency, and uniform model-tier selection is a separate, unrelated concern.

✔ **Correct** • Developer Productivity and Operational Enablement

Which of the following improve developer productivity when multiple teams are each building similar Claude-powered features? (Select all that apply.)

- ☒ **A** Keeping every team's prompts and evaluation results fully siloed from each other indefinitely.

- B** A shared, internal prompt and evaluation library teams can reuse and build on. Correct answer
- C** A shared internal SDK/wrapper that handles common integration boilerplate once. Correct answer
- D** Good local debugging tooling, such as request/response logging and replay capability. Correct answer

WHY

A shared prompt/eval library, a common SDK wrapper, and good local debugging tooling all reduce duplicated effort across teams. Keeping everything siloed is the pattern that causes the duplicated effort in the first place.



Want structured training before your exam? [Explore our AI Courses](#) →

Take your AI skills further with our Online AI Courses:

[AI Courses \(FDE, GenAI & Agentic AI Engineer, ML/AI Data Science & more\)](#) →

[AI Testing Courses \(AI & LLM Testing, Claude for Automation & more\)](#) →

ISHA TRAINING SOLUTIONS

Isha Training Solutions builds hands-on AI, GenAI, Agentic AI and LLM testing courses — including this free Claude Certified Architect – Professional mock exam platform with scenario-based practice questions and full-length scored mock exams.

QUICK LINKS

Claude Solutions Architect Course

Full CCA-F (Architect) Exam Prep

CCA-F Timed Mock Exams

CCAO-F (Associate) Exam Prep

CCDV-F (Developer) Exam Prep

AI Courses

Home

CONTACT

 **+91-9133190573**

ishatrainingsolutions.org

Isha Training Solutions © 2026. All rights reserved. · Claude Certified Architect – Professional mock exam prep · Independent, unofficial study material — not affiliated with, endorsed by, or sponsored by Anthropic.

Built by **Isha Training Solutions**